

Sorting

Summary of Sorting Algorithms and Concepts

Comparison-Based and Iterative Algorithms

1. Selection Sort

- **Description:** An iterative, comparison-based algorithm that repeatedly finds the **minimum** element from the unsorted portion of the array and places it at the beginning. It makes one swap per pass.
- **Time Complexity:** $O(n^2)$ (Worst, Average, Best)
- **Space Complexity:** $O(1)$ (In-place)

2. Bubble Sort

- **Description:** An iterative, comparison-based algorithm that repeatedly steps through the list, compares **adjacent elements**, and swaps them if they are in the wrong order, causing the largest elements to "bubble up" to their correct positions.
- **Time Complexity:** $O(n^2)$ (Worst, Average), $O(n)$ (Best, if already sorted)
- **Space Complexity:** $O(1)$ (In-place)

3. Insertion Sort

- **Description:** An iterative, comparison-based algorithm that builds the final sorted array one item at a time. It iterates through the input elements and **inserts** each element into its correct position among the already sorted elements.
- **Time Complexity:** $O(n^2)$ (Worst, Average), $O(n)$ (Best, if already sorted)
- **Space Complexity:** $O(1)$ (In-place)

Comparison-Based and Recursive Algorithms

1. Merge Sort

- **Description:** A recursive, comparison-based algorithm that follows the **Divide and Conquer** paradigm. It recursively divides the array into two halves, sorts them independently, and then **merges** the two sorted halves to produce the final result.
- **Time Complexity:** $O(n \log n)$ (Worst, Average, Best)
- **Space Complexity:** $O(n)$ (Requires auxiliary space for merging)

Merge Sort is $O(n \log n)$ because the algorithm divides the work into two factors:

1. **Height of Tree ($\log n$):** The array is repeatedly split in half, creating a recursion tree with a depth of $\log n$ levels. This represents the number of times the **Merge** operation must be performed along the path from the leaves up to the root.
2. **Work Done at Every Level (n):** At each of the $\log n$ levels, the total work required to merge all the sub-arrays back together is always $O(n)$ (linear time), because every element is processed exactly once in total across all merges

at that level.

The total time is the product of these factors: $\log n$ levels $\times O(n)$ work per level = $O(n \log n)$.

2. Quick Sort

- **Description:** A recursive, comparison-based algorithm also following the **Divide and Conquer** paradigm.

It works by picking one element, called the pivot, and then rearranging the other elements so that everything smaller than the pivot is on one side, and everything larger is on the other. This process is called partitioning. The pivot is now correctly placed. The algorithm then recursively applies the same logic to the two smaller piles on either side until the entire array is sorted.

- **Time Complexity:** $O(n^2)$ (Worst, if pivot selection is poor), $O(n \log n)$ (Average, Best)
- **Space Complexity:** $O(\log n)$ (Auxiliary space for recursion stack)

Non-Comparison Based

1. Counting sort

- **Mechanism:** A non-comparison, **stable** sort effective only for **integers** within a small range (k). It determines the output position of each element by **counting its occurrences** in the input.
- **Process:**
 1. Tally the **frequency** of each unique key in a temporary **Count Array**.
 2. Convert the Count Array into **cumulative counts** (prefix sums), which directly map to the final **starting index** of each key in the sorted output.
 3. Iterate backward through the input to place elements into the correct slot in the output array.
- **Complexity:**
 - Time: $O(n + k)$.
 - Space: $O(n + k)$.
- **Limitation:** Performance degrades significantly if the key range (k) is much larger than the number of elements (n).
- **Primary Use:** When k is small

1. Radix Sort

- **Description:** A non-comparison based algorithm that sorts integers by processing individual **digits** (or bits). It typically uses an intermediate stable sort (like Counting Sort) to sort the elements based on each digit, from least significant to most significant.
- **Time Complexity:** $O(d(n + b))$ (where d is the number of digits and b is the base)
- **Space Complexity:** $O(n + b)$ (Required for auxiliary counting/bucket arrays)

Comparison of Sort Algorithms and Concepts

1. Comparison of Sort Algorithms

- **In-place sort:** An algorithm that sorts the data using only a small, constant amount of **extra** memory (e.g., $O(1)$ or $O(\log n)$ auxiliary space). Examples include Selection Sort, Bubble Sort, Insertion Sort, and Quick Sort.
- **Stable sort:** An algorithm that preserves the **relative order of equal elements** from the input to the output. If two elements are equal, their order remains the same after sorting. Examples include Merge Sort, Insertion Sort, and Bubble Sort.

STL Implementation

1. `std::sort()` in C++ STL

- **Description:** The C++ Standard Library's `std::sort()` is implemented using **IntroSort**, which is a hybrid algorithm. It starts with **Quick Sort** for good average performance, switches to **Heap Sort** to guarantee $O(n \log n)$ worst-case complexity, and uses **Insertion Sort** for small partitions.
- **Time Complexity:** $O(n \log n)$ (Guaranteed)
- **Space Complexity:** $O(\log n)$ (Auxiliary space for recursion stack)
- **Note:** `std::sort()` is typically **not stable**. `std::stable_sort()` is used when stability is required.